Date: October 17, 2026

Artificial Intelligence - Lab

Lesson 7 - FINAL PROJECT: AI Virtual Waiter for Perpetual Bistro Using Ollama and Qwen

Final Project:

Final Project Specification: AI Virtual Waiter for Perpetual Bistro Using Ollama and Qwen

- Execution Window: October 19 – November 6, 2026
- Final Project Due Date: November 6, 2026
- Live Demo Presentations: November 7, 2026
- Core Tech Stack: Python 3.10+, Local Ollama (Qwen 2.5), SQLite, STT/TTS Speech Modules, Flask/CustomTkinter

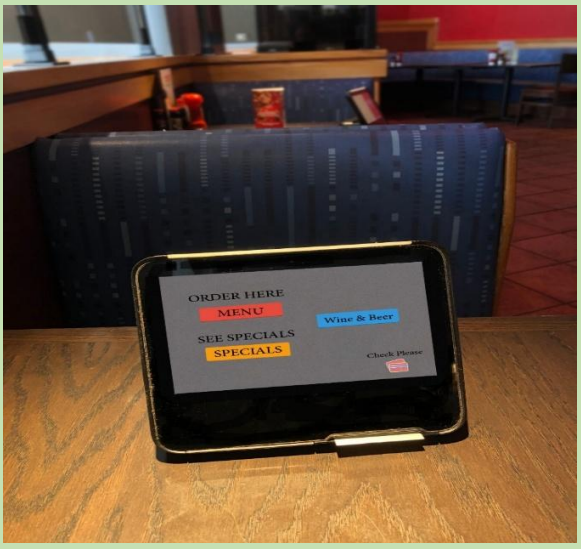
1. Business Scenario:

Project Overview & Business Scenario

Perpetual Bistro, the campus dining facility at UPHSD, seeks to modernize its customer service experience by deploying a table-side digital waiter. The system must run 100% locally (zero cloud dependencies) using Ollama (Qwen 2.5), Python, SQLite, and STT/TTS Speech Modules.

The AI Virtual Waiter serves as an interactive, voice-enabled, state-aware ordering assistant. Customers converse naturally—asking about menu ingredients, requesting customizations (*"Make it spicy, no onions"*), checking dietary restrictions, and managing their active order tab. The system must dynamically bind spoken or typed user intents to local SQLite tool functions, update real-time kitchen inventory, enforce strict Finite State Machine (FSM) state transitions, and generate a final itemized receipt upon checkout.

A sample image of the application Platform. Digital Tablet Ordering Interface Concept. Source: sabhu07 / Getty Images





Date: October 17, 2026

2. Learning Objectives:

By completing this Final project, you will be able to:

- **Knowledge (K) – Cognitive:**
    - Synthesize speech processing, agentic tool calling, relational database transactions, and Finite State Machines (FSM) into an integrated multi-modal AI agent.
    - Analyze edge-case dialogue failures (e.g., out-of-stock items, speech recognition errors, state-skipping attempts) to engineer programmatic guardrails.
- **Skills (S) – Psychomotor/Practical:**
    - Construct a voice-enabled I/O interface combining Speech-to-Text (STT) and Text-to-Speech (TTS) with an underlying stateful Python agent.
    - Implement dynamic agentic tool calling where Qwen 2.5 autonomously queries and mutates SQLite database tables (menu_items, table_sessions, cart_items).
    - Build a real-time ordering UI (Web Dashboard or Python GUI) that stays synchronized with database state changes and displays an active order tab.
- **Attitude (A) – Affective:**
    - Demonstrate professional engineering collaboration through driver-navigator tandem programming.
    - Value user-centric design principles, ensuring the AI agent remains courteous, transparent, and resilient during customer interactions.

3. Project Milestones & Execution Schedule:

Milestone	Deliverables	Due Date	Weight
Milestone 1	Voice I/O Pipeline, FSM Core Controller, and SQLite Tool Engine (restaurant_db.sql, db_tools.py, fsm_controller.py, speech_engine.py)	Wednesday, Oct 28, 2026	40%
Milestone 2	Integrated UI Dashboard, Guardrails, Final Codebase, Documentation, and Walkthrough Video (app.py, ui_dashboard/, documentation.md)	Saturday, Nov 06, 2026	60%
Live Defense	10-Minute Live Demonstration and Faculty Panel Defense	Saturday, Nov 7, 2026 *(Class Hours)*	*Final Exam*

Date: October 17, 2026

3. Required Tech Stack & Setup Rules

System Requirements & Environment:

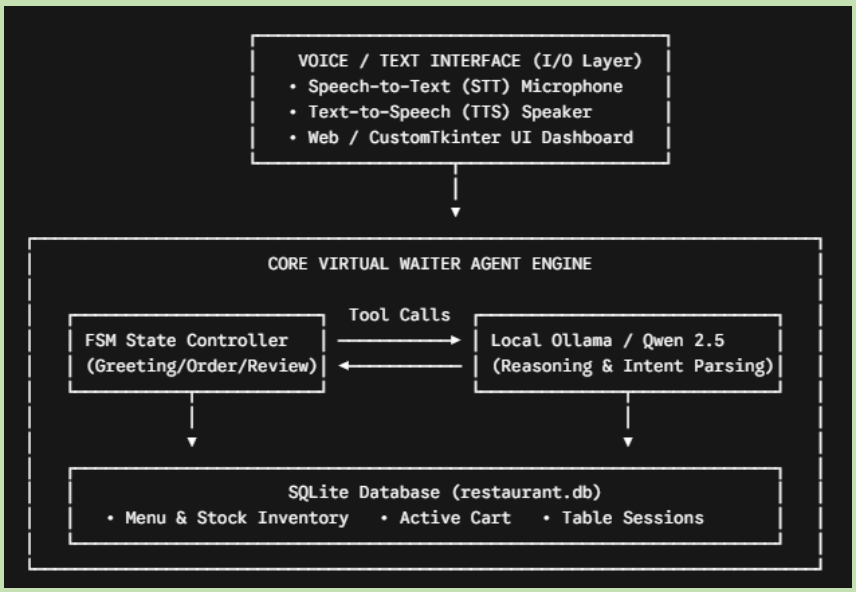
1. Local LLM Engine: Ollama running qwen2.5 (qwen2.5:7b recommended; qwen2.5:3b for 8GB RAM laptops).
2. Core Language & Database: Python 3.10+ and SQLite 3 (restaurant.db).
3. Voice/Speech Libraries: Speech-to-Text (e.g., whisper / speech_recognition) and Text-to-Speech (e.g., pyttsx3).
4. UI Dashboard Framework: Web GUI (Flask + HTML/Tailwind CSS) OR Desktop GUI (CustomTkinter).

4. Required File Architecture & Deliverables

Your final code submission must follow this folder structure:

```
ai_virtual_waiter/
│
├── init_restaurant_db.sql     # SQLite schema & seed dataset
├── init_db.py                 # DB initialization script
├── db_tools.py                # Tool functions & SQLite CRUD engine
├── fsm_controller.py          # Finite State Machine state manager
├── speech_engine.py           # Decoupled STT/TTS speech wrappers
├── app.py                     # Main application entry point
├── project_documentation.md   # Architecture, state diagrams & tandem log
│
└── ui_dashboard/              # Web or Desktop GUI files
    ├── templates/             # HTML templates (if using Flask)
    └── static/                # Assets & CSS styling
```

5. System Architecture & Conversation Flow



6. Live Defense Rules (November 7, 2026)

1. Hardware Execution: The system must run live on one of the pair's laptops without internet access or external cloud API keys.
2. Tandem Pair Defense: Both Driver and Navigator must participate and answer architectural questions.
3. The demo will test 2 edge-case scenarios:
    - Attempting to order an out-of-stock item (e.g., *Crispy Slice Salmon* when stock is 0).
    - Attempting to modify or cancel an order after reaching the CONFIRMED state.

7. Submission Instructions:

Submit a single zip file named [Student1_LastName]_[Student2_LastName]_FinalProject.zip under the Assignments tab in MS Teams. Ensure it contains your complete **codebase**, **database scripts**, **documentation markdown**, **and a link to your 3-minute video walkthrough**.

Here is the complete step-by-step workflow and implementation sequence for the Project Execution:

Phase 1: Environment Setup & Database Initialization (Oct 19 – Oct 22)

Foundational layer for local execution

1. **Initialize Project Directory: Create the standardized directory structure in VS Code:**

```
ai_virtual_waiter/
├── init_restaurant_db.sql
├── init_db.py
├── db_tools.py
├── fsm_controller.py
├── speech_engine.py
├── app.py
└── ui_dashboard/
```

2. **Pull Local LLM Model & Dependencies: Run the terminal setup commands to ensure qwen2.5 is active:**

Bash:

pip install pydantic jinja2 requests flask customtkinter speechrecognition pyttsx3

ollama pull qwen2.5

3. **Database Population: Run python init_db.py to create restaurant.db populated with the official Perpetual Bistro menu, category structure, and stock inventory.**

Date: October 17, 2026

Phase 2: Database Tooling & FSM State Machine (Oct 23 – Oct 25)

Building backend logic and state controls

1. Implement Database CRUD Wrapper (db_tools.py): Write Python helper functions for querying menu categories, calculating cart totals, and atomic stock subtractions:
    - get_menu_items(category)
    - add_item_to_order_tool(session_id, item_name, qty, instructions)
    - get_cart_summary(session_id)
2. **Construct FSM Controller (fsm_controller.py): Implement the state machine class to enforce strict dialogue states:**
    - GREETING →Captures table number.
    - ORDERING →Processes tool calls for cart modifications.
    - REVIEW →Presents itemized cart and subtotal for customer verification.
    - CONFIRMED →Commits transaction and locks cart modifications.

Phase 3: Decoupled Speech I/O Pipeline (Oct 26 – Oct 28)

Milestone 1 Deliverable Checkpoint

1. Build Speech Audio Wrappers (speech_engine.py): Implement input/output abstraction adapters:
    - Speech-to-Text (STT): Captures microphone audio using SpeechRecognition or whisper and converts to string text.
    - Text-to-Speech (TTS): Converts agent response strings to audio speech via pyttsx3.
2. Milestone 1 Verification & Submission: Run CLI integration script to verify that a spoken input triggers an FSM state transition and updates SQLite without GUI dependencies.

Milestone 1 Deadline: Wednesday, October 28, 2026

Phase 4: Agentic Tool Calling & Guardrails (Oct 29 – Nov 02)

Connecting Qwen 2.5 reasoning to database functions

1. Define JSON Tool Specs for Qwen: Inject available function signatures into Qwen system prompts so the local model emits structured JSON for tool calls.
2. Programmatic Inventory Guardrails: Enforce hard code guards inside db_tools.py so that if Qwen requests 5 orders of Pan-Seared Salmon but stock is 2, the system blocks the commit and prompts Qwen to offer alternatives.
3. Edge Case & Out-of-Domain Handling: Implement system prompt rules to politely reject off-topic questions (e.g., weather or general knowledge) and redirect the user back to ordering.

Phase 5: UI Dashboard Development & System Integration (Nov 03 – Nov 06)

Milestone 2 Deliverable Checkpoint

1. **Develop Interactive Interface (ui_dashboard/): Build the real-time ordering screen (Flask Web Interface or CustomTkinter Desktop App) displaying:**
    - Live Chat/Voice Transcript
    - Active Cart Table (Item, Qty, Instructions, Line Total)
    - Manual Human-in-the-Loop Override Buttons (+, -, Delete)
    - Grand Total and Checkout Confirmation Button
2. Main Application Entry Point (app.py): Chain all layers together: UI →Speech Engine →FSM Controller →Ollama/Qwen →SQLite Tools.
3. System Walkthrough Recording: Record a 3-5 minute video demonstrating a complete end-to-end voice ordering transaction from table greeting to receipt confirmation.

Milestone 2 & Final Code Deadline: Saturday, November 6, 2026

Phase 6: Live Demo Defense & Panel Evaluation (November 7, 2026)

Live presentation day

1. Pre-Defense Verification: Ensure local Ollama service is active and restaurant.db is reset to fresh inventory states.
2. Live Classroom Defense: Demonstrate a live voice interaction in front of the class and successfully navigate 2 edge-case stress tests (out-of-stock items and post-confirmation edit attempts).

Execution Summary Checklist for each group:

Task / File	**Driver Focus**	**Navigator Focus**	**Verification Test**
init_restaurant_db .sql	Writing DDL table structures	Reviewing foreign key constraints & seed data	Database builds without SQL errors via init_db.py.
db_tools.py	Coding SQLite CRUD functions	Writing inventory guard checks	add_item_to_order_tool() blocks items when stock_qty = 0.

Date: October 17, 2026

Task / File	Driver Focus	Navigator Focus	Verification Test
fsm_controller.py	Implementing state class logic	Verifying transition rules & guardrails	Cannot jump from GREETING directly to CONFIRMED.
speech_engine.py	Configuring STT microphone input	Tuning TTS voice speed & pitch	Microphones capture audio and speak back agent responses.
app.py & UI	Assembling main execution loop	Designing frontend CSS/Tkinter layout	Cart UI updates automatically when a tool call occurs.

Assessment Rubric (100 Points Total):

Criteria	Exemplary (90-100%)	Proficient (75-89%)	Developing (50-74%)	Unacceptable (<50%)	Weight
System Architecture & Integration	Flawlessly integrates Speech (STT/TTS), FSM state control, Ollama/Qwen, SQLite, and UI into a modular, responsive local application.	All modules integrate successfully; minor latency or audio stream buffering issues during transitions.	System functions in text mode, but speech integration or database sync fails frequently.	Pipeline crashes frequently; key modules fail to communicate.	30%
Agentic Tool Calling & Data Integrity	Qwen reliably triggers SQLite database tools. Inventory constraints, special instructions, and tab totals update with 100% accuracy.	Tool calling works accurately for standard orders; minor issues handling complex multi-item edits.	Tool calling frequently passes invalid arguments or ignores inventory stock limits.	No tool calling implemented; item data or prices are hallucinated by the LLM.	25%

Criteria	Exemplary (90-100%)	Proficient (75-89%)	Developing (50-74%)	Unacceptable (<50%)	Weight
FSM State Management & Guardrails	Strict state machine prevents illegal order shortcuts. Gracefully handles edge cases (out-of-stock items, cancellation, off-topic prompts).	FSM enforces primary workflow; minor edge cases allow user to bypass state checks occasionally.	FSM state breaks or resets mid-conversation when presented with unexpected inputs.	Stateless execution; treats every turn as an isolated query without session tracking.	20%
User Interface & Experience (UI/UX)	Professional UI dashboard clearly displays live chat transcript, itemized cart, subtotal/tax, and interactive checkout controls.	UI displays order cart and chat transcript cleanly, but lacks manual human-in-the-loop override buttons.	UI layout is unformatted or fails to update dynamically when database state changes.	No UI provided; operates purely through unformatted console logs.	15%
Live Defense & Tandem Collaboration	Pair demonstrates smooth, equal collaboration during live defense; answers panel technical questions with deep architectural insight.	System passes live demonstration; pair shows good division of labor with minor gaps in defense answers.	System fails during live demo edge cases; one student dominates defense explanations.	Pair is unable to demonstrate a working system live or answer basic architectural questions.	10%